Examining the use of DBNNs for normality prediction and statistical inference.

Ben A. Meaker

2026-11-14

Introduction

In the context of linear regression, the normality assumption of residuals is crucial for ensuring the validity of statistical inference, including confidence intervals for parameters of the model, prediction intervals for new responses, etc. Though regression modelling is robust against this assumption with larger sample sizes (i.e. n $>$ 200) through the Central Limit Theorem, with smaller to moderate sample sizes the assumption is a very important question, and often one with no clear answer.

Throughout the rest of this project, I will simply consider the assumption of normality on an arbitrary sample, though analyzing the normality of residuals is an important application.

The most commonly used diagnostic tools include visual diagnostics such as the QQPlot, which compares sample quantiles to theoretical normal quantiles and is implemented with qnorm(...) in R. Descriptive diagnostics include computing the kurtosis and skewness of the sample distribution, and checking for clear deviations from 0. Finally, if the former diagnostics are too inconclusive, one may consider formal diagnostic tests such as the Shapiro-Wilk test, which measures the deviation of the sample quantiles to theoretical quantiles and provides a reliable p-value for testing H_0: Sample is normally distributed vs H_A: Sample is not normally distributed. The test is especially powerful on skewed distributions and heavy-tailed distributions, such as the Cauchy distribution.

The Shapiro-Wilk test has its caveats however. The test tends to be oversensitive for large sample sizes to practically unimportant deviations from normality, is very sensitive to outliers, and often has low statistical power for small n, e.g., n $\in$ [10,20], meaning it often fails to detect sufficient evidence of non-normality when it is present.

Other formal diagnostic tests exists, though just like the Shapiro-Wilk test, they have their own case-specific strengths and drawbacks.

In this project, I hope to construct a non-parametric diagnostic test based on the learned "skill" of a model in identifying normal and non-normal distributions.

The natural choice for such a model would be an Artificial Neural Network (ANN), or in particular a Decision Based Neural Network (DBNN), but how would we obtain the data to train it? Luckily for us, generating pseudo random samples from a particular probability distribution is trivial, and is well supported by various programming languages, including R and Python. In this project, I will primarily be using Python for data generation, model construction and training, etc., and R for exploratory analysis and presentation. In particular, I will be using the PyTorch library in the implementation and training of the neural network, and the numpy library for data generation and statistical computation.

Modelling

The model will take a vector with 22 parameters: the sample size, the minimum and maximum value of the standardized sample, and the 0.05 through 0.95 quantiles of the standardized sample.

The model architecture was inspired by Miloš Simić, (arXiv:2009.13831, published in Neural Computing and Applications, 2021), and consists of 3 dense hidden layers of size 100, 20, and 10, and a single output.

The crude output is interpreted as the estimate of the log(odds) of the input sample being normally distributed. The loss function for training the model will be the Binary Cross Entropy with Logits Loss, or BCEWithLogitsLoss() in PyTorch. This loss function was chosen for its improved numerical stability over alternatives such as BCELoss() with manual Sigmoid activation.

The model will also utilize the Rectified Linear Unit activation function, a popular choice in Artificial Neural Networks, which is highly computationally efficient and improves the networks capability to learn complex, non-linear patterns in data. The latter especially will be crucial for our purposes, as the relationship between sample size & quantiles and normality is uncertain, if not unlikely, to follow a linear relationship.

Training

The training set will consist of an approximately 40%/60% split between normally distributed samples and non-normally distributed samples, with corresponding target labels 1 and 0. The split proportion is not implemented directly, and instead relies on random chance for distribution assignment. Since the sample size is so large, the true proportions will converge almost exactly to the theoretical proportions.

Various types of non-normal distributions will be included in the training set to improve model generalization and prevent over-fitting. The distributions are equally as probable and are used are as follows:

1) t-distribution with 1-15 degrees of freedom (including the Cauchy distribution)
2) Erlang distribution with 1-7 degrees of freedom (including the exponential distribution)
3) Uniform distribution
4) Multimodal distribution (mixture of normal distributions of different means)
5) Log-normal distribution
6) Variance mixture distribution (mixture of normal distributions of different standard deviations)

Distributions 1) and 5) are also equally likely to be reversed in orientation to change the skewness and improve generalization.

Since the samples are standardized before being fed into the model, the mean and standard deviation of the models by themselves are not influential, only the shape, kurtosis, skewness, etc., which the model will ideally learn to identify.

For each full iteration, approximately 100 000 standardized samples will be used for training. The proportion of sample sizes is specified explicitly beforehand, with around 36000 split evenly between sample sizes in [10,50] and the remaining 64000 split evenly in [55, 500]. I have decided to prioritize smaller sample sizes, since such samples are inherently more noisy and difficult to appropriately categorize.

The order of training for each sample size is randomly assigned to reduce over-fitting and improve training efficiency.

To avoid excessive memory usage, one full run of training data will be divided into 12 sub-iterations. The data from each sub-iteration is converted into a DataLoader object and shuffled with batch sizes of 128. Since data is not scarce (it is "synthetic data", generated computationally), I chose not to follow a traditional epoch based paradigm, and instead opted to use new data for each iteration. What this means is that it is impossible for the model to explicitly over fit to a particular piece or subset of data. Despite this, the model could possibly over fit to particular distributions shapes used in training (i.e. "identify" that a sample distribution is probably uniform because its seen so many similar samples during training).

We aren't too concerned about this though, since it is the entire purpose of the model to recognize when a sample is unlike a normal distribution and similar in some fashion to a common non-normal distribution it has seen during training. It is possible that the model will fail to generalize to novel, unseen sample distributions, though this is something that we will later test.

A fixed, universal validation set of size ~25000 is constructed with similar sample sizes proportions to the training set. The distribution of normal to non-normal distributions is also 40%/60%. The Laplace and Beta distributions are also included, and the Log-Normal distribution is excluded.

The model does not use the validation set for learning, only for evaluation. The validation set provides a good metric on how well the model predicts unseen data that can also differ somewhat from the typical training data (note the modifications to the possible distributions described before).

Training error is tracked over each sub-iteration and compared to the error on the validation set, and training is stopped when both measures plateau. This entire process can take several minutes, even on a powerful computer. Note that CUDA could be utilized for GPU acceleration of model training. Since that majority of computational resources are used for data generation, and the DBNN model is relatively small, I chose not to implement CUDA for simplicity.

(See training.py for full, detailed implementation)

Inference

The hypothesis we wish to test is this: Given a truly normally distributed sample, what is the probability that the model would classify it as non-normal? In other words, what is the FPR of the model? This depends both on the behavior of the model on normally distributed data, and the cutoff probability that we choose between 0 and 1, where close to 0 represents a near certain prediction of non-normality and close to 1 represents a near certain prediction of normality. If we want to limit the Type 1 Error to $\alpha = 0.05$, the risk that we reject H_0: Data is normally distributed when H_0 is in fact true, then must choose some cutoff $C \in (0, 1)$. This cutoff must ensure that if we looked at a large number of truly normally distributed samples S, only 0.05 or 5% of them would satisfy $p_{model}(S) < C$ This definition of risk is clearly following the frequentist interpretation of probability.

Unlike in many other areas of hypothesis testing however, we know nothing about the theoretical distribution of predictions/responses under the null hypothesis H_0. The DBNN is a "black-box" model after all. Instead, we will approximate a cutoff C for various sample sizes and choices of α through computational simulation. It is mandatory that we find cutoffs for each sample size, as the sample size is used as a parameter in the input vector, and very likely influences the models confidence in the normality or lack there-of in a sample. We should expect the cutoffs to on average increase with sample size, but we will explore this later. Essentially this is empirical calibration of the null distribution.

We can think of the cutoff at α to represent the quantile q_α of the distribution of model predictions on [0,1] under the null.

To ensure accurate cutoffs, we will use 25000 random normally distributed as our sample S and find q_α for $p_{model}(S)$. This process will be repeated for each desired sample size. As a necessary measure to save time, I will only be considering sample sizes starting at n=10 and increasing in increments of 5 to n=50, and then I will consider increments of 10 up until n=500. Because the model has only seen sample sizes up to and including 500, this is a natural stopping point. Later on, I will explore inference at higher sample sizes, e.g. n=1000, to check the model's generalization capability.

The approximate null quantiles are calculated and store in nullDistributionQuantiles.csv, each labeled with the associated significance level $\alpha \in \{0.05, 0.10\}$ and sample size.

(See normality_test_quantiles.py for full, detailed implementation)

```
quantiles <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\nullDistributionQu
head(quantiles)
```

```
##   SAMPLE_SIZE CRITICAL_ALPHA_05 CRITICAL_ALPHA_10
## 1          10            0.0492            0.0855
## 2          15            0.1030            0.1724
```

```
## 3          20            0.1567            0.2656
## 4          25            0.2078            0.3514
## 5          30            0.2949            0.4507
## 6          35            0.3352            0.4934
```

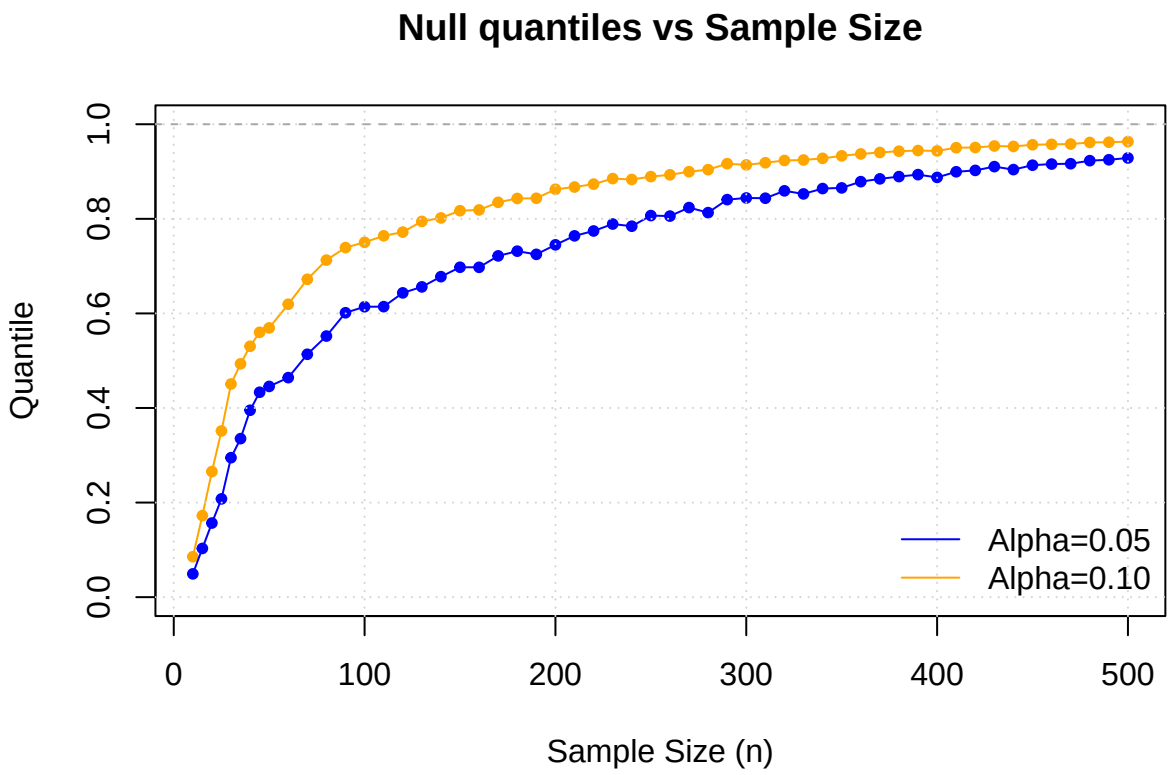
To understand how the critical values change with sample size, we will plot them and examine the trend.

```
plot(quantiles$SAMPLE_SIZE, quantiles$CRITICAL_ALPHA_05, col='blue', pch=19, cex=0.67, ylim=c(0.0, 1.00)
     xlab='Sample Size (n)', ylab='Quantile')

points(quantiles$SAMPLE_SIZE, quantiles$CRITICAL_ALPHA_10, col='orange', pch=19, cex=0.67)
lines(quantiles$SAMPLE_SIZE, quantiles$CRITICAL_ALPHA_05, col='blue')
lines(quantiles$SAMPLE_SIZE, quantiles$CRITICAL_ALPHA_10, col='orange')
grid()

legend(
  "bottomright",
  legend = c("Alpha=0.05", "Alpha=0.10"),
  col = c("blue", 'orange'),
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
)

abline(h=1.0, col='darkgrey', lty='dashed')
```



We can see the cutoff quantile increases very sharply from sample size n=10 to n=50, then increases steadily from n=50 onward. This shows that, as expected, the model becomes increasingly confident in its normality prediction as sample size increases, and minimally penalizes typical noise in larger samples. The $\alpha = 0.10$ curve is strictly greater than the $\alpha = 0.05$ curve, as expected.

Testing and statistical power

For our first batch of tests, we will compare the power of the DBNN model compared to the Shapiro-Wilk test for the t-distribution when holding the Type 1 error rate fixed at α=0.05.

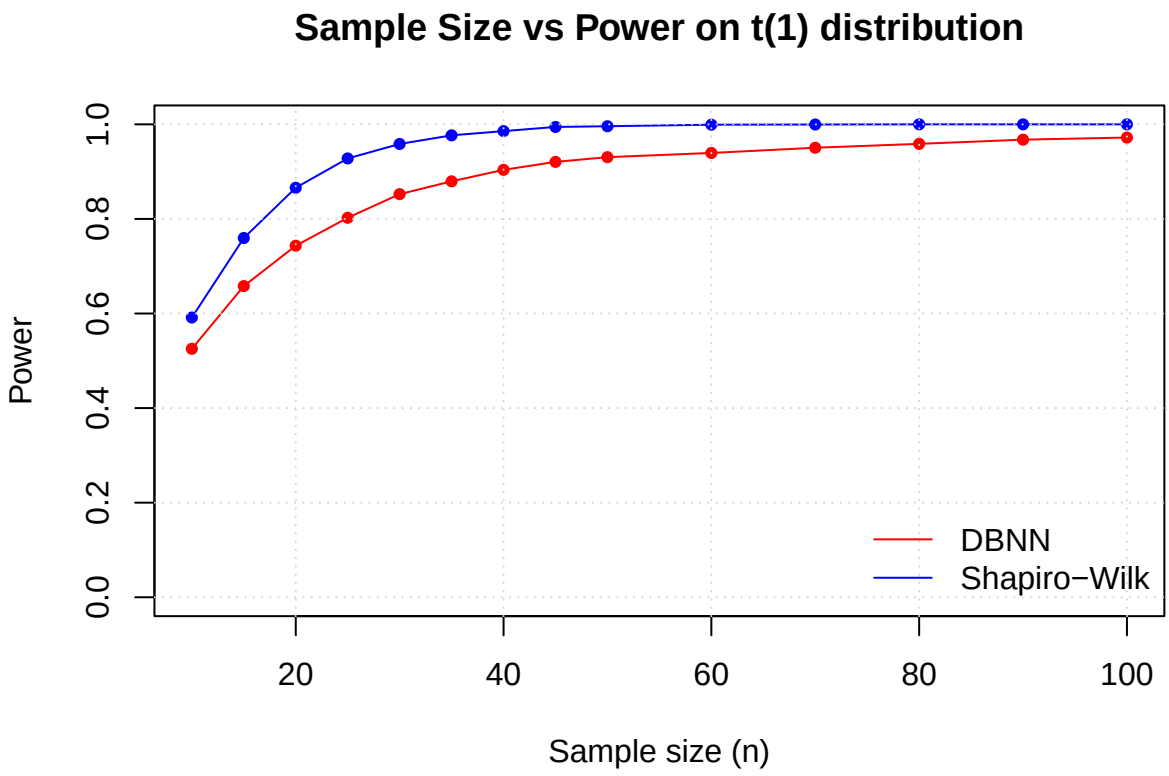
We will examine the findings for sample sizes of $n \in [10, 500]$ and degrees of freedom ranging from 1 to 15. The Python package 'scipy' was used to conduct the Shapiro-Wilk test. Both models are evaluated on the exact same collection of 5000 fixed samples per sample size.

(See testing.py for full, detailed implementation)

```
comparison_data <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\DBNN_VS_SW_t

plot(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red', main='Sample Size vs Power on
     pch=19, cex=0.7, xlab='Sample size (n)', ylab='Power')
points(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue', pch=19, cex=0.7)
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red')
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue')
grid()

legend(
  "bottomright",
  legend = c('DBNN', "Shapiro-Wilk"),
  col = c('red', 'blue'),
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
 )
```

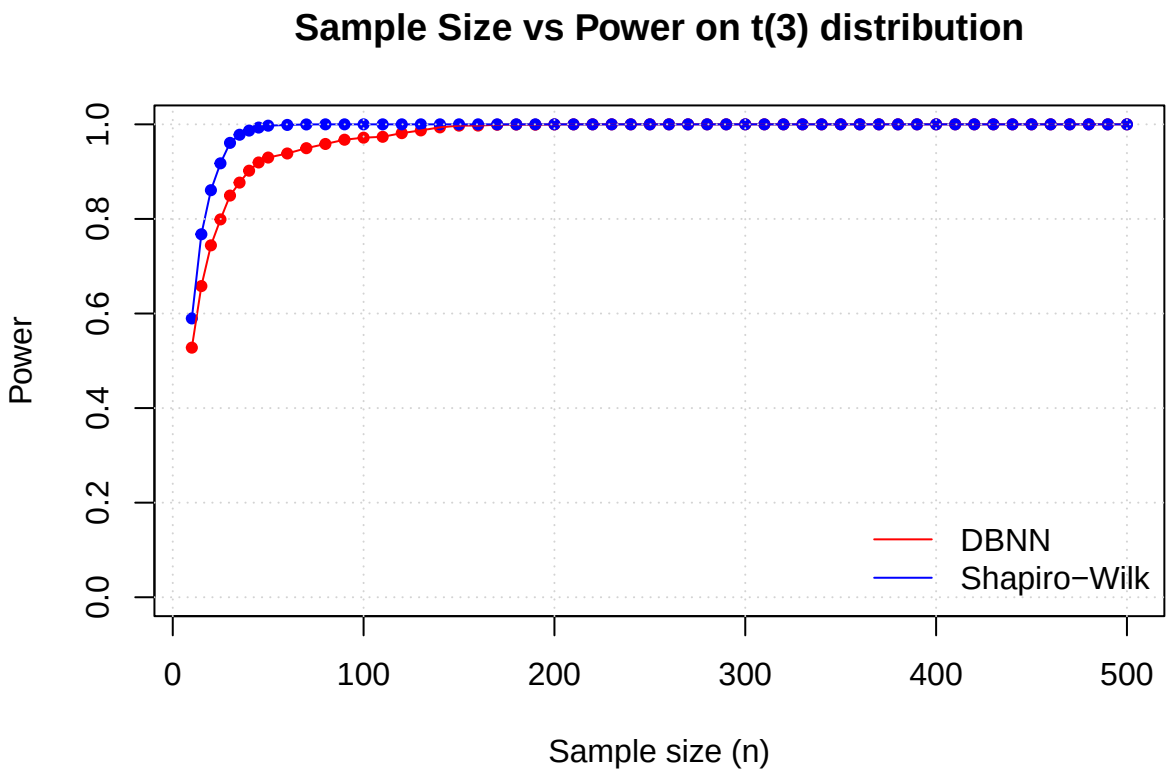


We can see that the Shapiro-Wilk test is considerably more powerful than our DBNN model at detecting non-normality in samples drawn from a Cauchy distribution.

```
comparison_data <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\DBNN_VS_SW_t3

plot(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red', main='Sample Size vs Power on t
     pch=19, cex=0.7, xlab='Sample size (n)', ylab='Power')
points(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue', pch=19, cex=0.7)
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red')
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue')
grid()

legend(
  "bottomright",
  legend = c('DBNN', "Shapiro-Wilk"),
  col = c('red', 'blue'),
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
 )
```

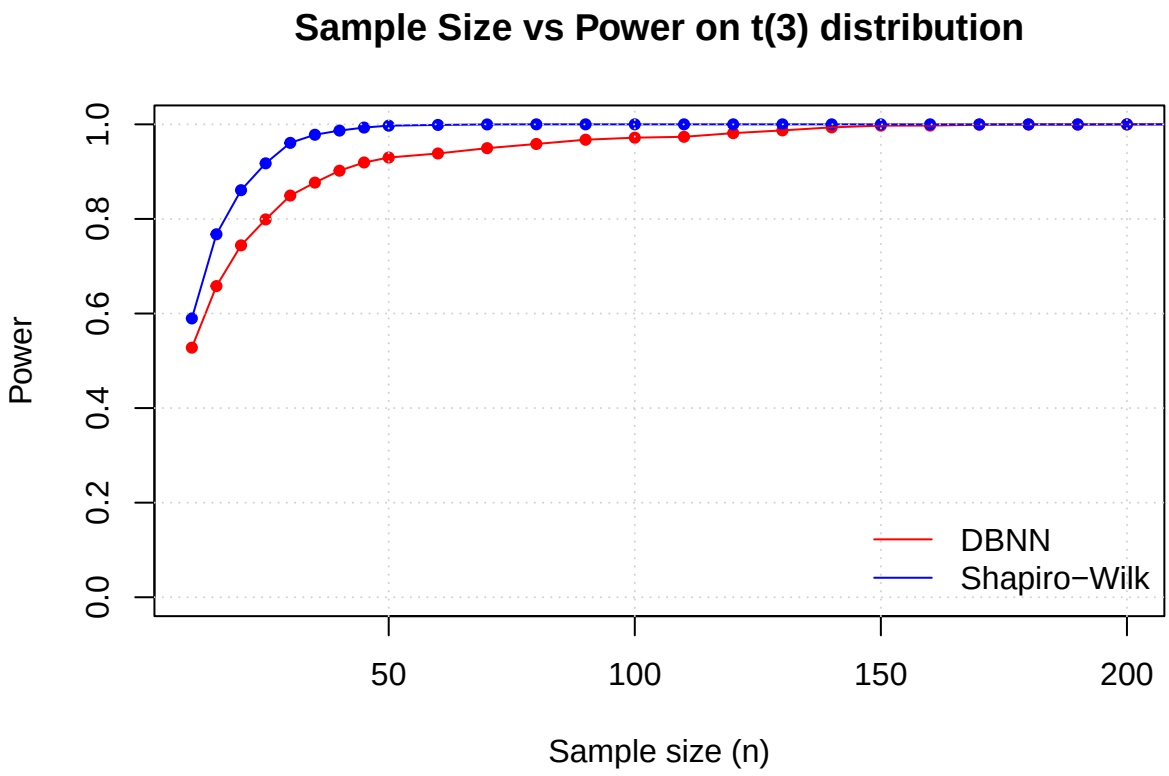


It still seems that the Shaprio-Wilk test is superior, let us now examine a shorter range of sample sizes.

```
comparison_data <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\DBNN_VS_SW_t3

plot(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red', main='Sample Size vs Power on t
     pch=19, cex=0.7, xlim=c(10,200), xlab='Sample size (n)', ylab='Power')
points(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue', pch=19, cex=0.7)
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red')
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue')
grid()

legend(
  "bottomright",
  legend = c('DBNN', "Shapiro-Wilk"),
  col = c('red', 'blue'),
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
 )
```

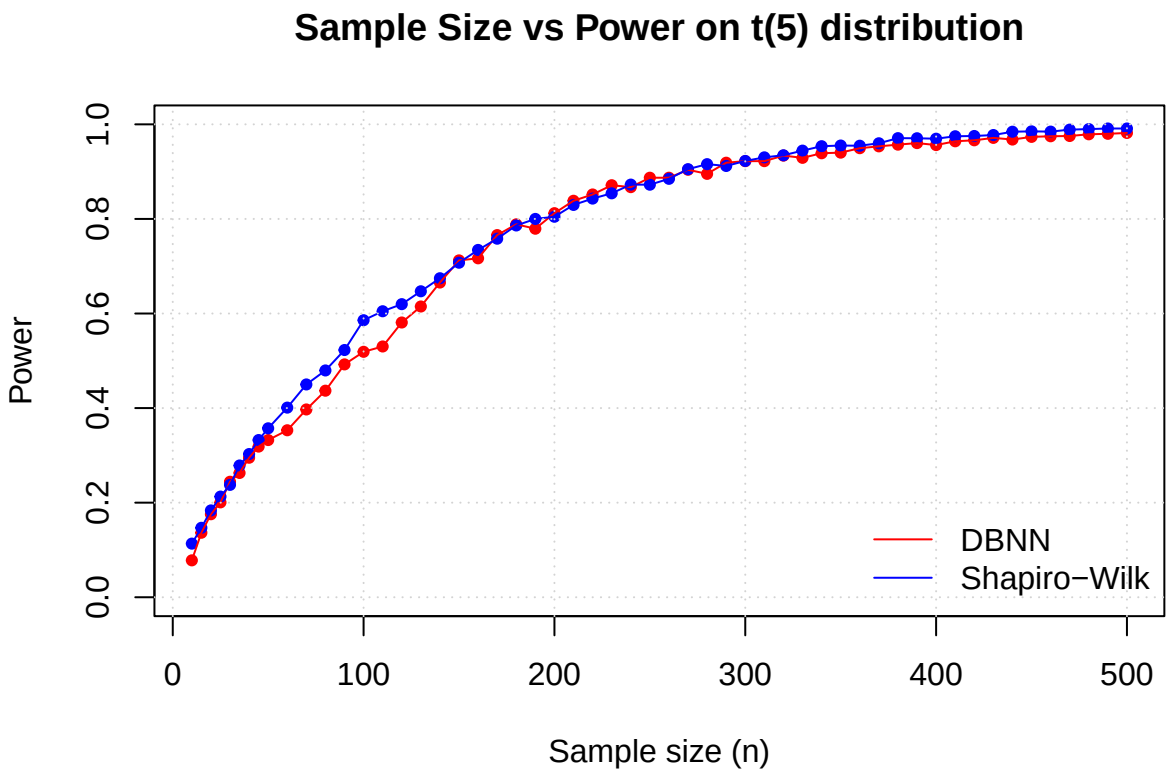


While both methods converge to a power of 1.0 by n=150, it seems that the Shapiro-Wilk test is especially more powerful for small-to-moderate sample sizes from around n=10 to n=80.

```
comparison_data <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\DBNN_VS_SW_t5

plot(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red', main='Sample Size vs Power on t
     pch=19, cex=0.7, xlab='Sample size (n)', ylab='Power')
points(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue', pch=19, cex=0.7)
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red')
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue')
grid()

legend(
  "bottomright",
  legend = c('DBNN', "Shapiro-Wilk"),
  col = c('red', 'blue'),
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
 )
```

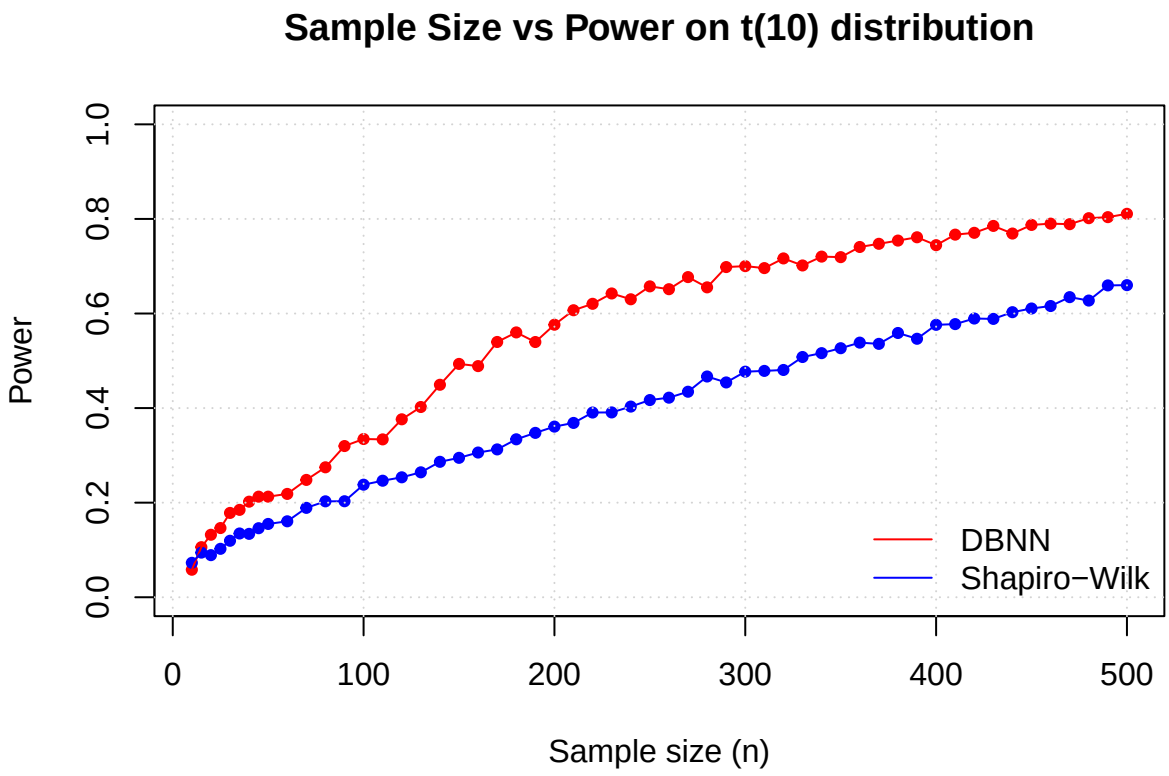


It appears that the power of the Shapiro-Wilk test and the DBNN model are near equivalent in terms of power.

```
comparison_data <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\DBNN_VS_SW_t

plot(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red', main='Sample Size vs Power on
     pch=19, cex=0.7, xlab='Sample size (n)', ylab='Power')
points(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue', pch=19, cex=0.7)
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red')
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue')
grid()

legend(
  "bottomright",
  legend = c('DBNN', "Shapiro-Wilk"),
  col = c('red', 'blue'),
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
 )
```

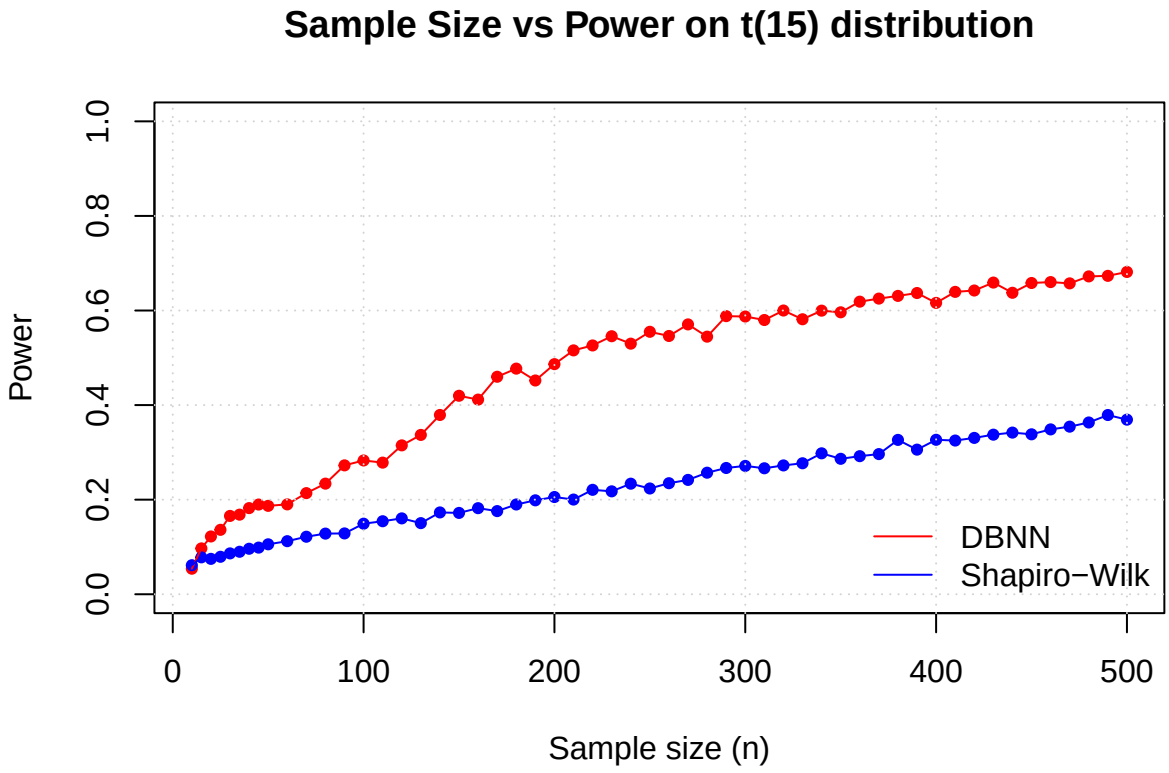


The DBNN is considerably more powerful, especially for n > 100.

```
comparison_data <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\DBNN_VS_SW_t

plot(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red', main='Sample Size vs Power on
     pch=19, cex=0.7, xlab='Sample size (n)', ylab='Power')
points(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue', pch=19, cex=0.7)
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red')
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue')
grid()

legend(
  "bottomright",
  legend = c('DBNN', "Shapiro-Wilk"),
  col = c('red', 'blue'),
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
 )
```



The DBNN is far more powerful than the Shapiro-Wilk test in this case, with almost 140% more power at n=200 ($1-\beta = 0.4866$ *vs* $1-\beta = 0.2056$)

This highlights a clear trend in the performance of the DBNN with regards to the Shapiro-Wilk test on the t-distribution. The Shapiro-Wilk test excels for the highly leptokurtic cases (high kurtosis) e.g., df=1 or df=3, it then sees rapid diminishing performance as the kurtosis decreases and the distribution converges to a normal distribution. The DBNN, having seen thousands of t-distributed samples during training, retains its ability to recognize overall shape and outperforms the Shapiro-Wilk test.

To show this more clearly, we will fix sample size at n=200 and then examine the trend in approximate power for varying degrees of freedom.

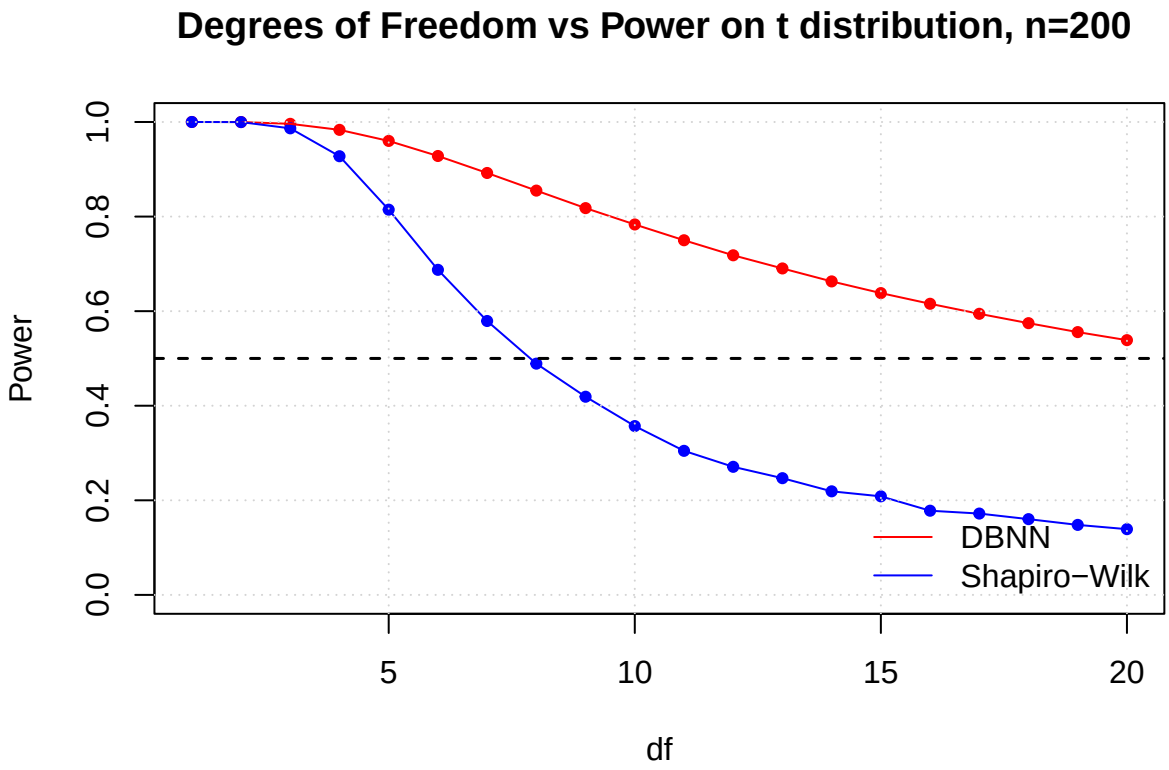
```
comparison_data <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\DBNN_VS_SW_t

plot(comparison_data$DF, comparison_data$POWER_DBNN, col='red',
     main='Degrees of Freedom vs Power on t distribution, n=200 ', ylim=c(0.0,1.0),
     pch=19, cex=0.7, xlab='df', ylab='Power')

points(comparison_data$DF, comparison_data$POWER_SW, col='blue', pch=19, cex=0.7)
lines(comparison_data$DF, comparison_data$POWER_DBNN, col='red')
lines(comparison_data$DF, comparison_data$POWER_SW, col='blue')
grid()
abline(h=0.5, col='black', lty='dashed', lwd=1.5)

legend(
  "bottomright",
  legend = c('DBNN', "Shapiro-Wilk"),
  col = c('red', 'blue'),
```

```
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
)
```



We can see that for n=200, the DBNN is far more powerful then the Shapiro-Wilk test in normality diagnostics, and its approximate power remains above $1 - \beta$=0.50 even for 20 degrees of freedom. It is well understood that the excess kurtosis of the t-distributions decreases rapidly as the degrees of freedom increases (as shown below), but can we conclude that the DBNN is also robust for other non-normal distributions with low kurtosis? For this, we will explore the quantitative relationship between excess kurtosis and power for various different mixed normal distributions, plotting the excess kurtosis of the sample distribution against the approximate powers for both tests, calculated using 1000 samples drawn from each distribution.

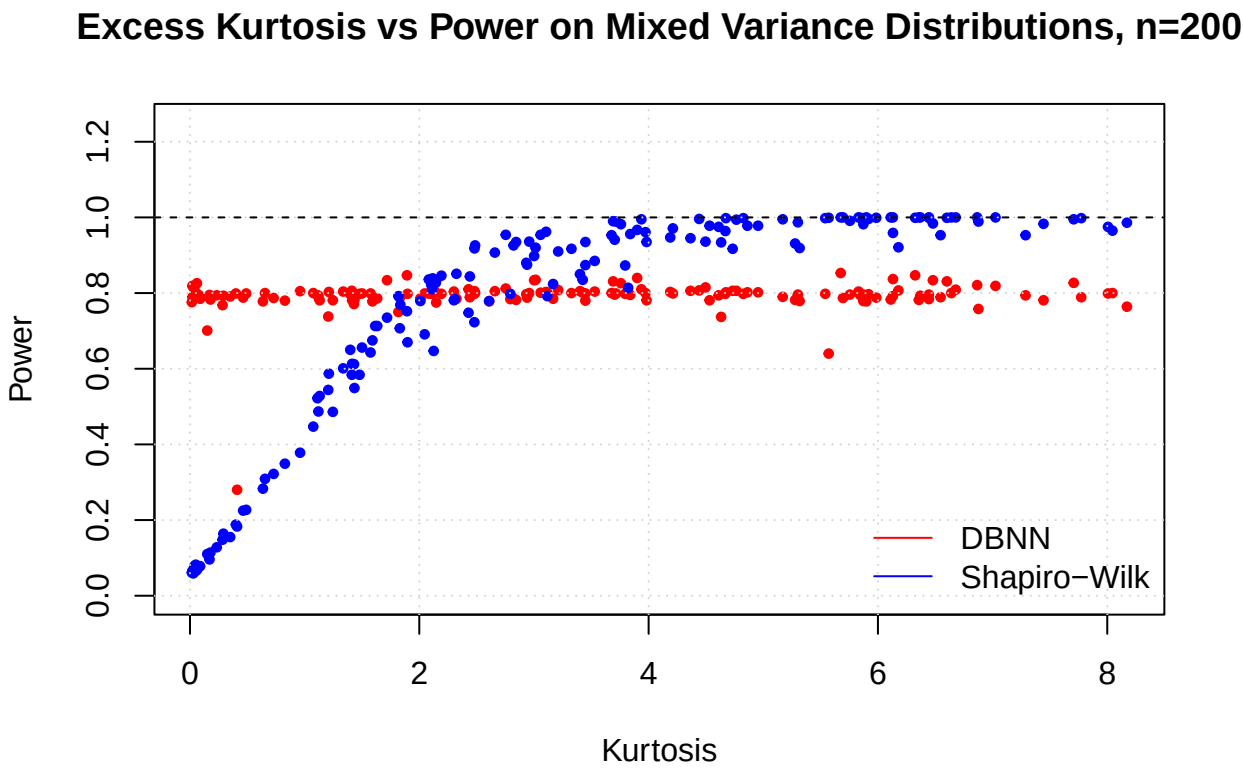
```
comparison_data <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\DBNN_VS_SW_mi

plot(comparison_data$KURTOSIS, comparison_data$POWER_DBNN, col='red', ylim=c(0,1.25),
     main='Excess Kurtosis vs Power on Mixed Variance Distributions, n=200',
     pch=19, cex=0.6, xlab='Kurtosis', ylab='Power')

points(comparison_data$KURTOSIS, comparison_data$POWER_SW, col='blue', pch=19, cex=0.6)

grid()
abline(h=1.0, col='black', lwd=1, lty=2)
legend(
  "bottomright",
```

```
  legend = c('DBNN', "Shapiro-Wilk"),
  col = c('red', 'blue'),
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
)
```



We can clearly see that our supposition holds on mixed variance normal distributions. The DBNN is highly robust to varying quantities of excess kurtosis for normality diagnostics, and the power of the test remains almost constant at 0.80. The Shapiro-Wilk test performs very poorly on the other hand for distributions with excess kurtosis nearing 0. This is to be expected, as the Shapiro-Wilk test is well documented to be most weak on symmetric, moderately leptokurtic distributions.

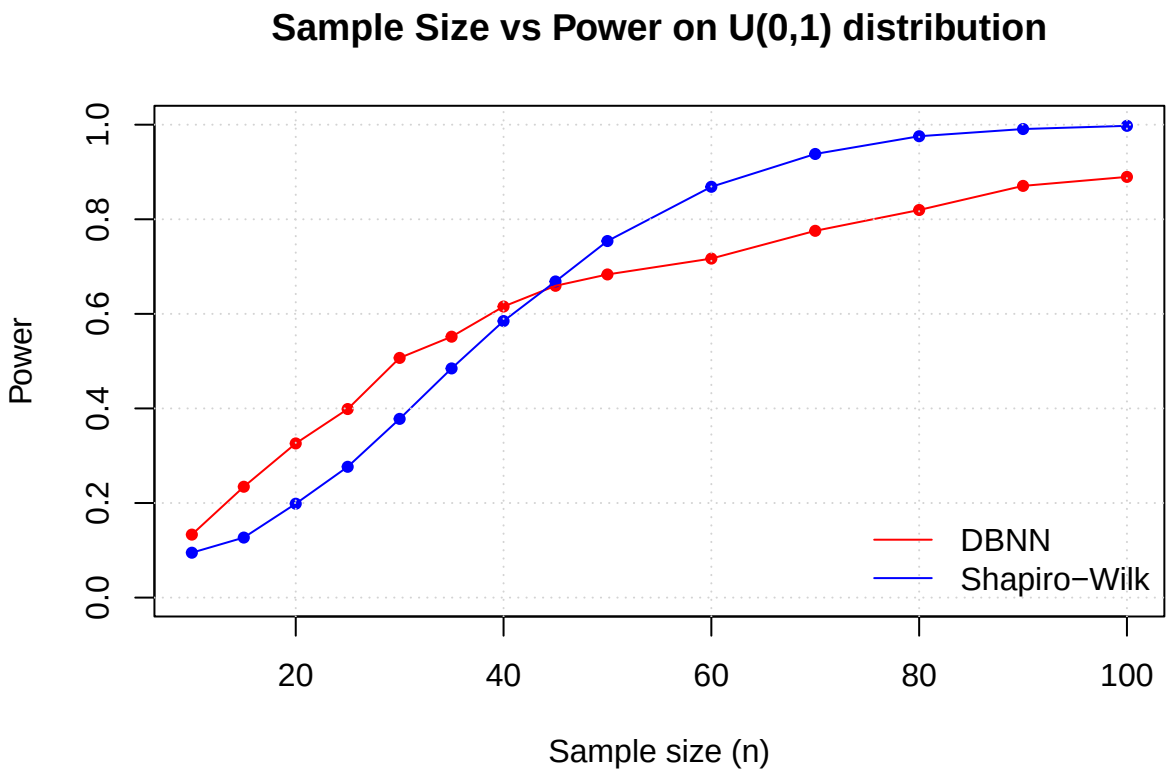
Though this is a substantial result, it is worth noting that the majority of cases in which normality should be tested, such as with residual errors, focuses mainly on large deviations from normality. A t(10) distribution for instance, though not truly normal, will often be "good enough" to assume to be normal. While it is theoretically an important result, it may not be too useful in real-world applications to flag such a distribution as non-normal. The main reason we would want a model that possesses high power on modestly leptokurtic distributions would be if we were constructing prediction intervals, or if we desired to conduct inference for model parameters at very small α.

Because both rely so heavily on the true tail behaviour of the underlying distribution, even a small difference between a normal distribution and the true distribution in this circumstance can render typical statistical inference misleading.

For our second test, we will compare the power of the DBNN model compared to the Shapiro-Wilk test for the uniform distribution holding the Type 1 error rate fixed at α=0.05.

```r
comparison_data <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\DBNN_VS_SW_u
plot(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red', main='Sample Size vs Power on
     pch=19, cex=0.7, xlab='Sample size (n)', ylab='Power')
points(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue', pch=19, cex=0.7)
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red')
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue')
grid()

legend(
  "bottomright",
  legend = c('DBNN', "Shapiro-Wilk"),
  col = c('red', 'blue'),
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
 )
```

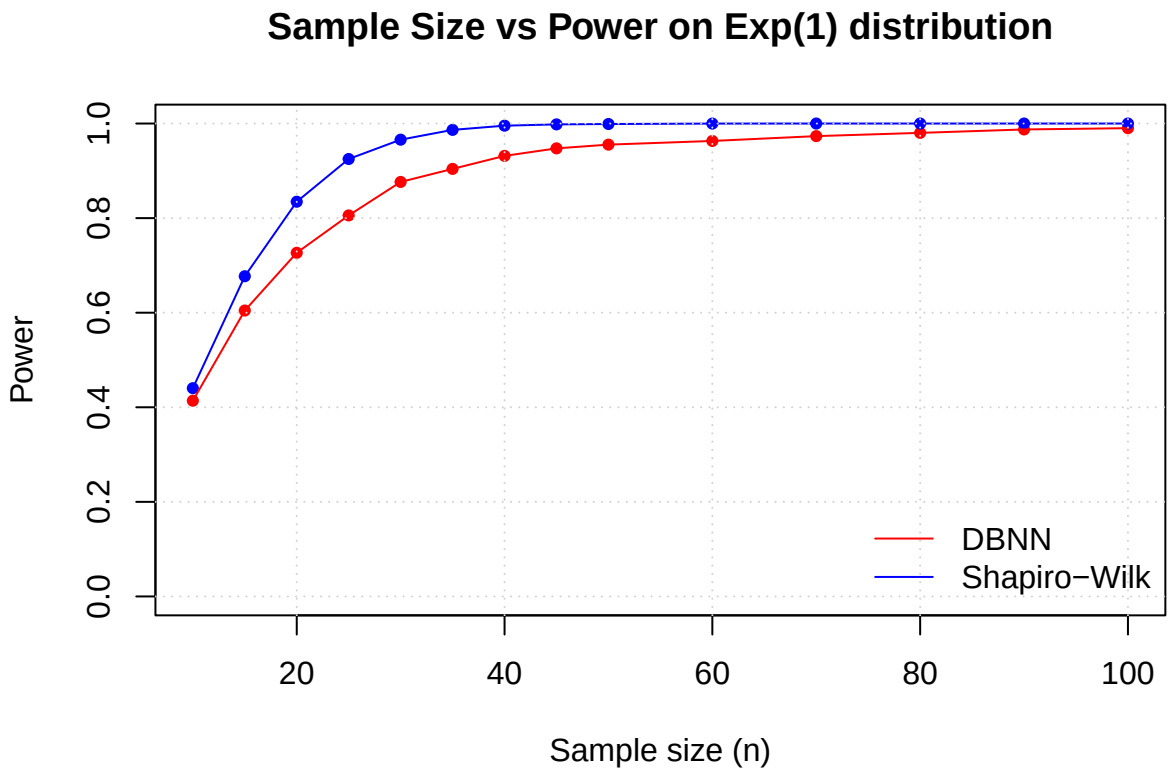


The DBNN model clearly exemplifies modestly higher statistical power for small sample size, but is surpassed by the Shapiro-Wilk test at n=45.

```r
comparison_data <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\DBNN_VS_SW_e
plot(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red', main='Sample Size vs Power on
     pch=19, cex=0.7, xlab='Sample size (n)', ylab='Power')
points(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue', pch=19, cex=0.7)
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red')
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue')
```

```
grid()

legend(
  "bottomright",
  legend = c('DBNN', "Shapiro-Wilk"),
  col = c('red', 'blue'),
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
 )
```

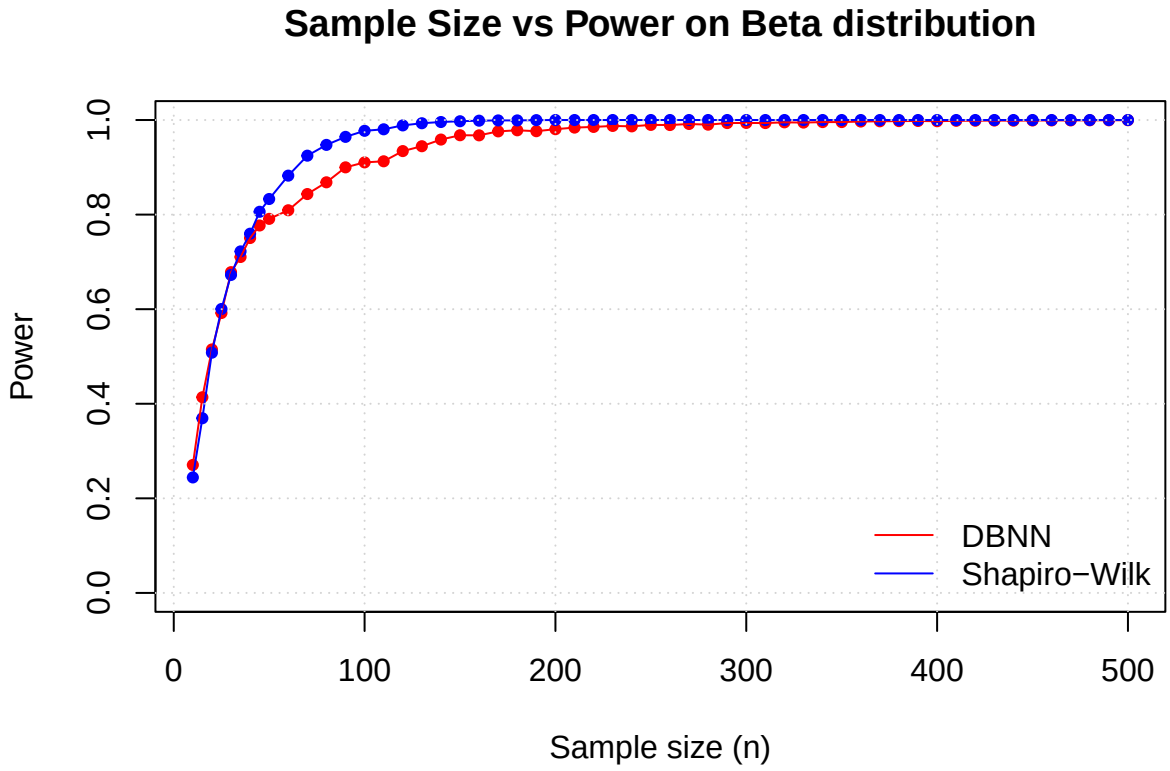


It appears that the Shapiro-Wilk test is modestly more powerful than the DBNN model on the standard exponential distribution. Give the Shapiro-Wilk test's known strength in detecting skewed sample distributions, this should not be too surprising.

Lastly, we will examine the behaviour of the DBNN model on a distribution unseen during training, the Beta distribution. The parameters used for the Beta distribution were randomized just as in the validation set, and can be examined in distributions.py

```
comparison_data <- read.csv("C:\\Users\\benme\\PycharmProjects\\DBNNproject\\testing_data\\DBNN_VS_SW_b
plot(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red', main='Sample Size vs Power on
     pch=19, cex=0.7, xlab='Sample size (n)', ylab='Power')
points(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue', pch=19, cex=0.7)
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_DBNN, col='red')
lines(comparison_data$SAMPLE_SIZE, comparison_data$POWER_SW, col='blue')
grid()
```

```r
legend(
  "bottomright",
  legend = c('DBNN', "Shapiro-Wilk"),
  col = c('red', 'blue'),
  lty = c(1, 1),
  lwd = 1,
  bty = "n"
 )
```



The Shapiro-Wilk test appears to be superior for n $>$ 50, though near equivalent in power to the DBNN for small sample sizes. Since the DBNN had never seen such a distribution shape during training, its performance provides good evidence that it generalizes well to new distributions.

Conclusion

The purpose of this project was not to create a replacement for formal tests such as the Shapiro-Wilk test. Our findings clearly indicate that the Shapiro-Wilk test is superior for detecting non-normality in skewed sample distributions and highly leptokurtic sample distributions. Instead, this project aimed to test the limits of the applications of machine learning in statistical hypothesis testing, and explore the gaps it could fill when compared with common parametric alternatives.

In our case, the DBNN model was much more powerful in identifying t-distributed samples of high degrees of freedom, as well as mixed variance normal distributions with low excess kurtosis, and could generalize well to previously unseen distributions. As discussed previously, this means it could be a valuable tool when results depend very carefully on the normality assumption and the tail behavior of the true underlying distribution.

All code and data associated with this project, including the trained DBNN model, has been included alongside this report.

For testing a given sample, one can use 'evaluation.py' for hypothesis testing. Note that the file only supports sample sizes from 10 to 500, as the model has not seen data beyond n=500 and OOD error is a serious issue. Consequently, null quantiles have not been calculated for larger n. If inference on larger samples is required, the training set can be easily modified to support larger sample sizes.